Advanced Analysis and Model Refinement: Telehealth Hypertension Analytics

Student: Raja Chakraborty **Course:** DSC-590: Data Science Capstone **Institution:** College of Science, Engineering, and Technology, Grand Canyon University **Instructor:** Jonathan Pollyn **Date:** September 30, 2026

This notebook is the executable companion to the written report. Every table, figure, and number in the report is produced by the cells below (fixed random seed, so results are reproducible). It covers:

1. A synthetic 24-hour ambulatory blood pressure monitoring (ABPM) cohort with half-hourly readings, measurement noise, artifacts, missing readings, and a simulated office spot-check.
2. Benchmarks: an office-reading AHA rule, a guideline ABPM-threshold rule, logistic regression, and default XGBoost.
3. Three documented refinements (feature engineering, cross-validated hyperparameter search, isotonic calibration) evaluated on an untouched 30% holdout.
4. Exploratory analysis of circadian profiles, morning surge, office-versus-ABPM discordance, feature attributions, and boundary errors.

1. Environment setup

In [1]:

```
import json, time, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import loguniform, randint, uniform
from sklearn.base import clone
from sklearn.calibration import CalibratedClassifierCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, average_precision_score, confusion_matrix,
                             f1_score, precision_recall_fscore_support, roc_auc_score, roc_curve)
from sklearn.model_selection import (RandomizedSearchCV, RepeatedStratifiedKFold,
                                     StratifiedKFold, cross_val_score, train_test_split)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import xgboost as xgb
import shap

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
OUT = "."  # figures and metrics are written next to the notebook

STAGES = ["Normal", "Elevated", "Stage 1", "Stage 2"]
STAGE_COLORS = ["#8fb8ea", "#4f8fdc", "#2a5fae", "#14306b"]  # ordinal: light to dark
BEFORE, AFTER = "#eb6834", "#2a78d6"
plt.rcParams.update({"font.family": "sans-serif", "font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "axes.grid": True, "grid.color": "#e6e6e3",
                     "grid.linewidth": 0.6, "axes.edgecolor": "#8a8984", "figure.dpi": 110,
                     "savefig.dpi": 200, "savefig.bbox": "tight", "legend.frameon": False})
```

```
metrics = {}
print("xgboost", xgb.__version__, "| shap", shap.__version__)
```

```
xgboost 2.0.3 | shap 0.49.1
```

2. Synthetic ABPM cohort

Real patient records are protected under HIPAA, so the cohort is simulated. Compared with the earlier phase (which generated one summary row per patient), each patient now has **48 half-hourly readings** over 24 hours, which makes circadian analysis possible. Generator assumptions:

- **Label** = the patient's adjudicated stage, i.e. their *long-run* blood pressure state (450 patients per stage, matching earlier phases).
- **Day-to-day variability:** the recorded day differs from the long-run state by N(0, 5) mmHg systolic, so a single monitoring day is an imperfect window on the true stage.
- **Sleep and wake:** diary bedtime ~N(22.5 h, 0.6) and wake time ~N(6.5 h, 0.6); nocturnal dip ratio varies by stage (normal dippers 12–18%, Stage 2 from -2% to 6%).
- **Morning surge:** a post-waking rise whose amplitude grows modestly with stage, an assumption drawn from Kario et al. (2003).
- **Measurement:** reading noise SD 6/4 mmHg, 3% cuff/motion artifacts of ±20–40 mmHg, 8% missing readings.
- **Office spot-check:** one clinic reading including a white-coat effect ~N(+4, 7) mmHg, so some patients read high in clinic (white-coat) and some read low (masked).

Because the morning surge and dipping differences are *built into the generator*, any analysis that "finds" them is checking that the pipeline works. It is not a clinical discovery. The report discusses this.

In [2]:

```
N_PER = 450
spec = {  # long-run daytime SBP/DBP mean, sd; dip range; surge mean
    0: dict(sbp=(112, 5), dbp=(74, 4), dip=(0.12, 0.18), surge=8),
    1: dict(sbp=(124, 3), dbp=(76, 3), dip=(0.08, 0.14), surge=10),
    2: dict(sbp=(134, 4), dbp=(84, 4), dip=(0.04, 0.10), surge=13),
    3: dict(sbp=(156, 12), dbp=(98, 8), dip=(-0.02, 0.06), surge=17),
}
t = np.arange(0, 24, 0.5)  # 48 half-hourly slots
sig = lambda x: 1 / (1 + np.exp(-x))

rows, readings = [], []
pid = 0
for stage, s in spec.items():
    for _ in range(N_PER):
        sbp_true = rng.normal(*s["sbp"]); dbp_true = rng.normal(*s["dbp"])
        day_eff = rng.normal(0, 5)
        dip = rng.uniform(*s["dip"])
        surge = max(0, rng.normal(s["surge"], 5))
        bed, wake = rng.normal(22.5, 0.6), rng.normal(6.5, 0.6)
        hr_mean = rng.normal(74, 8)
        # smooth asleep indicator (1 = asleep), wrapping midnight
        tt = np.where(t < 12, t + 24, t)
        asleep = sig((tt - bed) * 3) * sig((wake + 24 - tt) * 3)
        surge_curve = surge * np.exp(-((t - (wake + 1.5)) ** 2) / (2 * 1.2 ** 2))
```

```
        sbp_level = sbp_true + day_eff
        dbp_level = dbp_true + 0.6 * day_eff
        sbp = sbp_level * (1 - dip * asleep) + surge_curve + rng.normal(0, 6, t.size)
        dbp = dbp_level * (1 - dip * asleep) + 0.5 * surge_curve + rng.normal(0, 4, t.size)
        hr = hr_mean * (1 - 0.10 * asleep) + rng.normal(0, 4, t.size)
        art = rng.random(t.size) < 0.03
        sbp[art] += rng.choice([-1, 1], art.sum()) * rng.uniform(20, 40, art.sum())
        miss = rng.random(t.size) < 0.08
        sbp[miss] = dbp[miss] = hr[miss] = np.nan
        wce = rng.normal(4, 7)
        rows.append(dict(pid=pid, target=stage, age=float(np.clip(rng.normal(56, 12), 21, 88)),
                         bmi=float(np.clip(rng.normal(27.5, 4.5), 18.5, 42)),
                         sex=rng.choice(["F", "M"]), wake=wake, bed=bed,
                         office_sbp=sbp_level + wce + rng.normal(0, 6),
                         office_dbp=dbp_level + 0.5 * wce + rng.normal(0, 4)))
        readings.append(pd.DataFrame(dict(pid=pid, hour=t, sbp=sbp, dbp=dbp, hr=hr,
                                          awake=asleep < 0.5)))
        pid += 1

patients = pd.DataFrame(rows)
abpm = pd.concat(readings, ignore_index=True)
print(f"{len(patients):,} patients, {len(abpm):,} half-hourly readings "
      f"({abpm.sbp.isna().mean():.1%} missing)")
patients.groupby("target").size().rename(index=dict(enumerate(STAGES)))
```

```
1,800 patients, 86,400 half-hourly readings (7.9% missing)
```

Out[2]:
```
target
Normal      450
Elevated    450
Stage 1     450
Stage 2     450
dtype: int64
```

3. Feature extraction from the 24-hour record

Summary features (the earlier phase's inputs) are computed from the noisy readings using the patient's diary sleep/wake times. Morning-surge features follow Kario et al. (2003): **sleep-trough surge** = mean SBP in the 2 h after waking minus the mean of the lowest night reading and its two neighbours; **pre-waking surge** = mean SBP in the 2 h after waking minus the mean in the 2 h before waking. Because both conventional definitions include the night-to-day transition, they also absorb nocturnal dipping, so a third, **dip-adjusted surge excess** = mean SBP in the 2 h after waking minus the patient's daytime mean is also computed.

In [3]:
```
def summarize(g, wake):
    day, night = g[g.awake], g[~g.awake]
    hrs_after = (g.hour - wake) % 24
    morning = g[hrs_after < 2].sbp.mean()
    prewake = g[(hrs_after >= 22)].sbp.mean()
    ns = night.sbp.dropna().values
    k = int(np.nanargmin(ns)) if ns.size else 0
    trough = ns[max(0, k - 1):k + 2].mean() if ns.size else np.nan
    return pd.Series(dict(sbp_day=day.sbp.mean(), dbp_day=day.dbp.mean(),
                          sbp_night=night.sbp.mean(), dbp_night=night.dbp.mean(),
                          heart_rate=g.hr.mean(), surge_sleep_trough=morning - trough,
                          surge_prewake=morning - prewake,
                          surge_excess=morning - day.sbp.mean()))

feats = abpm.groupby("pid").apply(lambda g: summarize(g, patients.wake.iloc[g.name]))
df = patients.join(feats, on="pid")
```

```
def engineer(d):
    d = d.copy()
    d["map_day"] = d.dbp_day + (d.sbp_day - d.dbp_day) / 3
    d["map_night"] = d.dbp_night + (d.sbp_night - d.dbp_night) / 3
    d["pulse_pressure"] = d.sbp_day - d.dbp_day
    d["dip_ratio"] = (d.sbp_day - d.sbp_night) / d.sbp_day
    return d

df = engineer(df)
RAW = ["sbp_day", "dbp_day", "sbp_night", "dbp_night", "heart_rate", "age", "bmi"]
ENG = RAW + ["map_day", "map_night", "pulse_pressure", "dip_ratio"]
SURGE = ENG + ["surge_sleep_trough", "surge_prewake", "surge_excess"]
df[RAW + ["dip_ratio", "surge_sleep_trough"]].describe().round(1)
```

Out[3]:

	sbp_day	dbp_day	sbp_night	dbp_night	heart_rate	age	bmi	dip_ratio	surg
count	1800.0	1800.0	1800.0	1800.0	1800.0	1800.0	1800.0	1800.0	
mean	133.3	83.7	121.8	76.7	71.6	55.9	27.7	0.1	
std	19.0	11.7	22.9	14.1	7.9	12.2	4.5	0.0	
min	93.3	57.7	78.0	48.1	43.4	21.0	18.5	-0.0	
25%	119.4	75.3	104.0	66.1	66.6	47.4	24.5	0.1	
50%	130.2	81.0	118.4	73.3	71.6	56.3	27.6	0.1	
75%	143.2	90.2	135.6	85.8	76.8	64.0	30.8	0.1	
max	200.9	131.8	192.9	129.5	104.0	88.0	41.9	0.2	

4. Holdout split and evaluation helpers

A stratified 30% holdout (540 patients) is set aside before any tuning or calibration and is used only for final evaluation. All model selection uses cross-validation on the 70% training set. (The previous version of this notebook fitted the calibrator on the test set, which leaks information. That is fixed here.)

In [4]:

```
train_idx, test_idx = train_test_split(df.index, test_size=0.30, stratify=df.target,
                                       random_state=RANDOM_STATE)
tr, te = df.loc[train_idx], df.loc[test_idx]
y_tr, y_te = tr.target.values, te.target.values
Y_te = np.eye(4)[y_te]

def ece_top_label(y, P, bins=10):
    conf, pred = P.max(1), P.argmax(1)
    edges = np.linspace(0, 1, bins + 1)
    e = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            e += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return e

def evaluate(name, pred, P=None):
    p, r, f, _ = precision_recall_fscore_support(y_te, pred, labels=range(4), zero_division=0)
    cm = confusion_matrix(y_te, pred, labels=range(4))
```

```
    spec_ = [(cm.sum() - cm[i].sum() - cm[:, i].sum() + cm[i, i]) / (cm.sum() - cm[i].sum())
             for i in range(4)]
    out = dict(name=name, accuracy=accuracy_score(y_te, pred), macro_f1=f.mean(),
               crisis_recall=r[3], precision=p.tolist(), recall=r.tolist(), f1=f.tolist(),
               specificity=spec_, cm=cm.tolist())
    if P is not None:
        out.update(roc_auc=[roc_auc_score(Y_te[:, i], P[:, i]) for i in range(4)],
                   pr_auc=[average_precision_score(Y_te[:, i], P[:, i]) for i in range(4)],
                   brier=float(np.mean(np.sum((P - Y_te) ** 2, axis=1))),
                   ece=ece_top_label(y_te, P))
        out["macro_roc_auc"] = float(np.mean(out["roc_auc"]))
    return out

def latency_ms(fn, X, n=300):
    row = X.iloc[[0]]
    fn(row)
    ts = []
    for i in range(n):
        r = X.iloc[[i % len(X)]]
        t0 = time.perf_counter(); fn(r); ts.append((time.perf_counter() - t0) * 1000)
    return float(np.median(ts))

def boot_ci(pred, stat, n=1000, seed=RANDOM_STATE):
    g = np.random.default_rng(seed); vals = []
    for _ in range(n):
        i = g.integers(0, len(y_te), len(y_te))
        vals.append(stat(y_te[i], pred[i]))
    return [float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))]
```

5. Benchmarks

- **Office AHA rule:** 2017 ACC/AHA office thresholds applied to a single clinic reading (Whelton et al., 2018). Guidelines actually recommend averaging two or more readings on two or more occasions, so this is a deliberately simple stand-in for spot-check practice.
- **ABPM guideline rule:** daytime ABPM thresholds that correspond to the office categories (120/80, 130/80, 135/85; Whelton et al., 2018, Table 11) applied to the 24-hour record. This is the strongest non-ML benchmark because it uses the same data the model sees.
- **Logistic regression** (standardized raw features) and **default XGBoost** (raw features).

In [5]:

```
def aha_rule(s, d, cut2=(140, 90), cut1=(130, 80)):
    return np.select([(s >= cut2[0]) | (d >= cut2[1]), (s >= cut1[0]) | (d >= cut1[1]),
                      (s >= 120) & (d < 80)], [3, 2, 1], 0)

pred_office = aha_rule(te.office_sbp.values, te.office_dbp.values)
pred_abpm = aha_rule(te.sbp_day.values, te.dbp_day.values, cut2=(135, 85))

lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
lr.fit(tr[RAW], y_tr)
xgb_default = xgb.XGBClassifier(random_state=RANDOM_STATE, eval_metric="mlogloss")
xgb_default.fit(tr[RAW], y_tr)

res = {}
res["office_rule"] = evaluate("Office AHA rule (single reading)", pred_office)
res["abpm_rule"] = evaluate("ABPM guideline rule (daytime mean)", pred_abpm)
res["logreg"] = evaluate("Logistic regression", lr.predict(te[RAW]), lr.predict_proba(te[RAW]))
P_default = xgb_default.predict_proba(te[RAW])
res["xgb_default"] = evaluate("Default XGBoost", P_default.argmax(1), P_default)
res["office_rule"]["latency_ms"] = res["abpm_rule"]["latency_ms"] = latency_ms(
    lambda r: aha_rule(r.sbp_day.values, r.dbp_day.values), te)
res["logreg"]["latency_ms"] = latency_ms(lambda r: lr.predict_proba(r[RAW]), te)
```

```
res["xgb_default"]["latency_ms"] = latency_ms(lambda r: xgb_default.predict_proba(r[RAW]), te)

pd.DataFrame(res).T[["accuracy", "macro_f1", "crisis_recall", "macro_roc_auc", "brier", "ece",
                     "latency_ms"]].astype(float).round(3)
```

Out[5]:

	accuracy	macro_f1	crisis_recall	macro_roc_auc	brier	ece	latency_ms
office_rule	0.543	0.521	0.956	NaN	NaN	NaN	0.224
abpm_rule	0.615	0.588	0.993	NaN	NaN	NaN	0.224
logreg	0.883	0.884	0.948	0.98	0.182	0.056	2.345
xgb_default	0.889	0.889	0.970	0.98	0.181	0.076	2.904

6. Iterative refinement

Refinement 1: feature engineering. Add mean arterial pressure (day and night), pulse pressure, and the nocturnal dipping ratio. Default XGBoost hyperparameters are kept, so the change is isolated to features.

In [6]:

```
xgb_fe = xgb.XGBClassifier(random_state=RANDOM_STATE, eval_metric="mlogloss")
xgb_fe.fit(tr[ENG], y_tr)
P_fe = xgb_fe.predict_proba(te[ENG])
res["xgb_fe"] = evaluate("+ Feature engineering", P_fe.argmax(1), P_fe)
res["xgb_fe"]["latency_ms"] = latency_ms(lambda r: xgb_fe.predict_proba(r[ENG]), te)
{k: round(res["xgb_fe"][k], 3) for k in ["accuracy", "macro_f1", "crisis_recall", "brier", "ece"]}
```

Out[6]:

```
{'accuracy': 0.889,
 'macro_f1': 0.889,
 'crisis_recall': 0.97,
 'brier': 0.176,
 'ece': 0.065}
```

Refinement 2: hyperparameter search. Randomized search (Bergstra & Bengio, 2012), 40 candidates, 5-fold stratified cross-validation on the training set only, optimizing multi-class log loss (a proper scoring rule, so the search rewards good probabilities rather than only correct labels).

In [7]:

```
search = RandomizedSearchCV(
    xgb.XGBClassifier(random_state=RANDOM_STATE, eval_metric="mlogloss"),
    {"max_depth": randint(2, 8), "learning_rate": loguniform(0.01, 0.3),
     "n_estimators": randint(100, 600), "subsample": uniform(0.6, 0.4),
     "colsample_bytree": uniform(0.5, 0.5), "min_child_weight": randint(1, 10),
     "reg_lambda": loguniform(0.1, 10)},
    n_iter=40, scoring="neg_log_loss", cv=StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE),
    random_state=RANDOM_STATE, n_jobs=1)
search.fit(tr[ENG], y_tr)
xgb_tuned = search.best_estimator_
best_params = {k: (round(float(v), 4) if isinstance(v, float) else int(v))
               for k, v in search.best_params_.items()}
```

```
P_tuned = xgb_tuned.predict_proba(te[ENG])
res["xgb_tuned"] = evaluate("+ Hyperparameter search", P_tuned.argmax(1), P_tuned)
res["xgb_tuned"]["latency_ms"] = latency_ms(lambda r: xgb_tuned.predict_proba(r[ENG]), te)

# Fold-to-fold stability before vs after tuning (5x3 repeated CV, training set only)
rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
cv_before = cross_val_score(clone(xgb_fe), tr[ENG], y_tr, cv=rcv, scoring="neg_log_loss", n_jobs=1)
cv_after = cross_val_score(clone(xgb_tuned), tr[ENG], y_tr, cv=rcv, scoring="neg_log_loss", n_jobs=1)
cvf_before = cross_val_score(clone(xgb_fe), tr[ENG], y_tr, cv=rcv, scoring="f1_macro", n_jobs=1)
cvf_after = cross_val_score(clone(xgb_tuned), tr[ENG], y_tr, cv=rcv, scoring="f1_macro", n_jobs=1)
metrics["tuning"] = dict(best_params=best_params,
                         cv_logloss_before=[float(-cv_before.mean()), float(cv_before.std())],
                         cv_logloss_after=[float(-cv_after.mean()), float(cv_after.std())],
                         cv_f1_before=[float(cvf_before.mean()), float(cvf_before.std())],
                         cv_f1_after=[float(cvf_after.mean()), float(cvf_after.std())])
best_params, metrics["tuning"]
```

Out[7]:

```
({'colsample_bytree': 0.9989,
  'learning_rate': 0.0248,
  'max_depth': 3,
  'min_child_weight': 2,
  'n_estimators': 319,
  'reg_lambda': 0.1268,
  'subsample': 0.7115},
 {'best_params': {'colsample_bytree': 0.9989,
   'learning_rate': 0.0248,
   'max_depth': 3,
   'min_child_weight': 2,
   'n_estimators': 319,
   'reg_lambda': 0.1268,
   'subsample': 0.7115},
  'cv_logloss_before': [0.44537788262331635, 0.057357438684505616],
  'cv_logloss_after': [0.3017228770861584, 0.038086399429530286],
  'cv_f1_before': [0.8676185355787175, 0.01427946113935369],
  'cv_f1_after': [0.8741498693033566, 0.016756141190511744]})
```

Refinement 3: isotonic calibration (tested). `CalibratedClassifierCV` with 5-fold internal cross-validation on the **training set**: each fold's tuned model is calibrated on data it did not see during fitting. The holdout is never touched. The step is kept only if it improves holdout Brier score and ECE without an unacceptable latency cost.

In [8]:

```
xgb_cal = CalibratedClassifierCV(clone(xgb_tuned), method="isotonic", cv=5)
xgb_cal.fit(tr[ENG], y_tr)
P_cal = xgb_cal.predict_proba(te[ENG])
res["xgb_cal"] = evaluate("+ Isotonic calibration", P_cal.argmax(1), P_cal)
res["xgb_cal"]["latency_ms"] = latency_ms(lambda r: xgb_cal.predict_proba(r[ENG]), te)

explainer = shap.TreeExplainer(xgb_tuned)
res["xgb_tuned"]["shap_latency_ms"] = latency_ms(lambda r: explainer.shap_values(r[ENG]), te, n=100)

order = ["office_rule", "abpm_rule", "logreg", "xgb_default", "xgb_fe", "xgb_tuned", "xgb_cal"]
summary = pd.DataFrame([res[k] for k in order]).set_index("name")[
    ["accuracy", "macro_f1", "crisis_recall", "macro_roc_auc", "brier", "ece", "latency_ms"]]
summary.astype(float).round(3)
```

Out[8]:

name	accuracy	macro_f1	crisis_recall	macro_roc_auc	brier	ece	latency_ms
Office AHA rule (single reading)	0.543	0.521	0.956	NaN	NaN	NaN	0.224
ABPM guideline rule (daytime mean)	0.615	0.588	0.993	NaN	NaN	NaN	0.224
Logistic regression	0.883	0.884	0.948	0.980	0.182	0.056	2.345
Default XGBoost	0.889	0.889	0.970	0.980	0.181	0.076	2.904
+ Feature engineering	0.889	0.889	0.970	0.980	0.176	0.065	3.854
+ Hyperparameter search	0.902	0.902	0.970	0.984	0.148	0.014	3.547
+ Isotonic calibration	0.900	0.900	0.970	0.982	0.152	0.024	22.673

Decision. Because the hyperparameter search optimized log loss, the tuned model was already well calibrated. Isotonic calibration added five model copies (about 6x the inference time) and slightly *worsened* holdout Brier score and ECE, so it is **not adopted**. The refined model carried forward is the feature-engineered, tuned XGBoost.

Bootstrap confidence intervals (1,000 resamples of the holdout)

In [9]:

```
crisis_rec = lambda y, p: ((p == 3) & (y == 3)).sum() / max((y == 3).sum(), 1)
macro_f1 = lambda y, p: f1_score(y, p, average="macro")
ci = {}
for k, pred in {"abpm_rule": pred_abpm, "logreg": lr.predict(te[RAW]),
                "xgb_default": P_default.argmax(1), "xgb_tuned": P_tuned.argmax(1)}.items():
    ci[k] = dict(accuracy=boot_ci(pred, accuracy_score), macro_f1=boot_ci(pred, macro_f1),
                 crisis_recall=boot_ci(pred, crisis_rec))
# paired difference: calibrated pipeline minus each comparator
g = np.random.default_rng(RANDOM_STATE)
for ref, pref in [("xgb_default", P_default.argmax(1)), ("abpm_rule", pred_abpm),
                  ("logreg", lr.predict(te[RAW]))]:
    d = []
    for _ in range(1000):
        i = g.integers(0, len(y_te), len(y_te))
        d.append(accuracy_score(y_te[i], P_tuned.argmax(1)[i]) - accuracy_score(y_te[i], pref[i]))
    ci[f"acc_diff_vs_{ref}"] = [float(np.percentile(d, 2.5)), float(np.percentile(d, 97.5))]
metrics["ci"] = ci
ci
```

Out[9]:

```
{'abpm_rule': {'accuracy': [0.575925925925926, 0.6592592592592592],
  'macro_f1': [0.552563332262609, 0.6287493703819215],
  'crisis_recall': [0.9765578863188976, 1.0]},
 'logreg': {'accuracy': [0.8555555555555555, 0.9074074074074074],
  'macro_f1': [0.8550237919937655, 0.9081715818584591],
  'crisis_recall': [0.9078924038820456, 0.979173801369863]},
 'xgb_default': {'accuracy': [0.8592592592592593, 0.9148148148148149],
  'macro_f1': [0.8608690314422773, 0.9139171735355074],
  'crisis_recall': [0.9379723837209303, 0.9929589776420762]},
 'xgb_tuned': {'accuracy': [0.8740277777777778, 0.9259259259259259],
  'macro_f1': [0.874094444339795, 0.9262202001275329],
  'crisis_recall': [0.937984496124031, 0.9929601917057903]},
 'acc_diff_vs_xgb_default': [-0.005555555555555538, 0.031481481481481555],
 'acc_diff_vs_abpm_rule': [0.24259259259259258, 0.32777777777777783],
 'acc_diff_vs_logreg': [-0.0018518518518518823, 0.03703703703703698]}
```


Figure 1: Confusion matrices before and after refinement

In [10]:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4.2))
for ax, key, title in [(axes[0], "xgb_default", "Before: default XGBoost"),
                       (axes[1], "xgb_tuned", "After: refined model (engineered + tuned)")]:
    cm = np.array(res[key]["cm"])
    ax.imshow(cm / cm.sum(1, keepdims=True), cmap="Blues", vmin=0, vmax=1)
    for i in range(4):
        for j in range(4):
            v = cm[i, j] / cm[i].sum()
            ax.text(j, i, f"{cm[i, j]}\n({v:.0%})", ha="center", va="center", fontsize=9,
                    color="white" if v > 0.55 else "#0b0b0b")
    ax.set_xticks(range(4), STAGES); ax.set_yticks(range(4), STAGES)
    ax.set_xlabel("Predicted stage"); ax.set_ylabel("True stage"); ax.grid(False)
    ax.set_title(f"{title}\naccuracy {res[key]['accuracy']:.1%}", fontweight="bold")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig1_confusion.png"); plt.show()
```

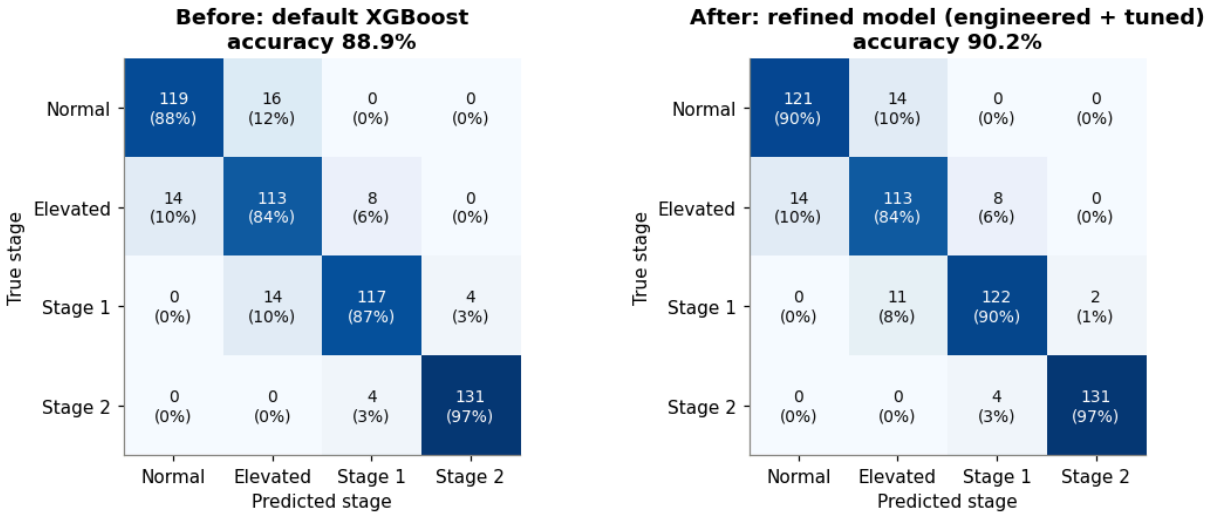


Figure 2: Metric progression across refinement stages

In [11]:

```
steps = ["xgb_default", "xgb_fe", "xgb_tuned", "xgb_cal"]
labels = ["Default\nXGBoost", "+ Feature\nengineering", "+ Hyperparameter\nsearch", "+ Isotonic calib.\n(tested, reject
ed)"]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4))
x = np.arange(4); w = 0.26
for k, (m, lab, c) in enumerate([("accuracy", "Accuracy", "#2a78d6"),
                                 ("macro_f1", "Macro F1", "#1baf7a"),
                                 ("crisis_recall", "Stage 2 recall", "#eb6834")]):
    vals = [res[s][m] * 100 for s in steps]
    a1.bar(x + (k - 1) * w, vals, w * 0.92, label=lab, color=c)
    for xi, v in zip(x + (k - 1) * w, vals):
        a1.text(xi, v + 0.3, f"{v:.1f}", ha="center", fontsize=7.5, color="#52514e")
lo = min(res[s][m] for s in steps for m in ["accuracy", "macro_f1", "crisis_recall"]) * 100
a1.set_ylim(np.floor(lo) - 4, 102.5); a1.set_xticks(x, labels, fontsize=8.5)
a1.set_ylabel("Holdout score (%)"); a1.legend(loc="upper left", ncol=3, fontsize=8)
a1.set_title("Classification metrics (higher is better)", fontweight="bold")
for k, (m, lab, c) in enumerate([("brier", "Brier score", "#4a3aa7"), ("ece", "ECE", "#e87ba4")]):
    vals = [res[s][m] for s in steps]
    a2.bar(x + (k - 0.5) * 0.36, vals, 0.33, label=lab, color=c)
    for xi, v in zip(x + (k - 0.5) * 0.36, vals):
        a2.text(xi, v + 0.002, f"{v:.3f}", ha="center", fontsize=7.5, color="#52514e")
a2.set_xticks(x, labels, fontsize=8.5); a2.set_ylabel("Error (lower is better)")
a2.legend(fontsize=8); a2.set_title("Probability quality (lower is better)", fontweight="bold")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig2_progression.png"); plt.show()
```

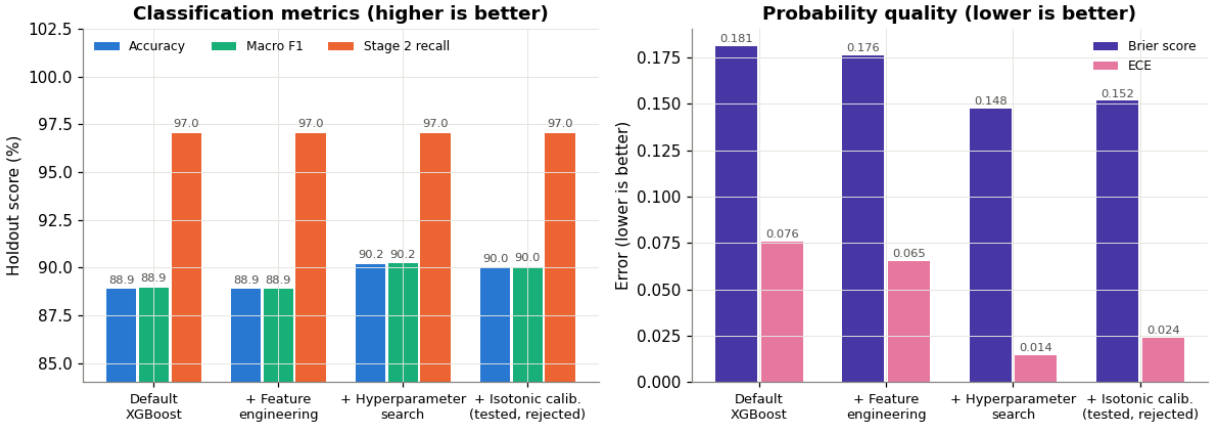


Figure 3: One-vs-rest ROC curves before and after refinement

In [12]:

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.4), sharey=True)
for ax, P, key, title in [(axes[0], P_default, "xgb_default", "Before: default XGBoost"),
                          (axes[1], P_tuned, "xgb_tuned", "After: refined model (engineered + tuned)")]:
    for i in range(4):
        fpr, tpr, _ = roc_curve(Y_te[:, i], P[:, i])
        ax.step(fpr, tpr, where="post", lw=2, color=STAGE_COLORS[i],
                label=f"{STAGES[i]} (AUC {res[key]['roc_auc'][i]:.3f})")
    ax.plot([0, 1], [0, 1], ls="--", color="#8a8984", lw=1, label="Chance (AUC 0.5)")
    ax.set_xlim(-0.01, 1); ax.set_ylim(0, 1.01); ax.set_xlabel("False positive rate")
    ax.set_title(f"{title}\nmacro AUC {res[key]['macro_roc_auc']:.3f}", fontweight="bold")
    ax.legend(loc="lower right", fontsize=8)
    ins = ax.inset_axes([0.42, 0.42, 0.5, 0.33])
    for i in range(4):
        fpr, tpr, _ = roc_curve(Y_te[:, i], P[:, i])
        ins.step(fpr, tpr, where="post", lw=1.5, color=STAGE_COLORS[i])
    ins.set_xlim(0, 0.2); ins.set_ylim(0.6, 1.0); ins.tick_params(labelsize=7)
    ins.set_title("zoom: FPR 0-0.2", fontsize=7)
axes[0].set_ylabel("True positive rate")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig3_roc.png"); plt.show()
```

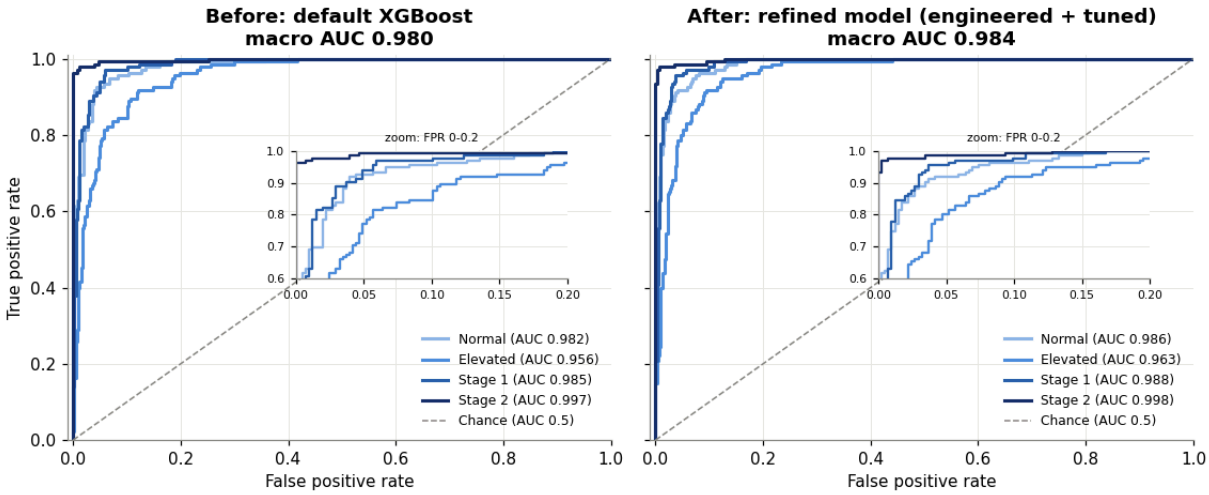


Figure 4: Reliability diagram (top-label confidence) across refinement stages

In [13]:

```
def reliability(P, bins=10):
    conf, ok = P.max(1), (P.argmax(1) == y_te)
    edges = np.linspace(0, 1, bins + 1); xs, ys, ns = [], [], []
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.sum() >= 5:
            xs.append(conf[m].mean()); ys.append(ok[m].mean()); ns.append(m.sum())
    return np.array(xs), np.array(ys), np.array(ns)

fig, (ax, axh) = plt.subplots(2, 1, figsize=(6.4, 6), height_ratios=[3, 1], sharex=True)
ax.plot([0, 1], [0, 1], ls="--", color="#8a8984", lw=1, label="Perfect calibration")
for P, key, c, mk, lab in [(P_default, "xgb_default", BEFORE, "s", "Default XGBoost"),
                           (P_tuned, "xgb_tuned", AFTER, "o", "Tuned (log-loss objective)"),
                           (P_cal, "xgb_cal", "#4a3aa7", "^", "Tuned + isotonic")]:
    xs, ys, _ = reliability(P)
    ax.plot(xs, ys, marker=mk, color=c, lw=2, ms=7,
            label=f"{lab} (ECE {res[key]['ece']:.3f}, Brier {res[key]['brier']:.3f})")
    axh.hist(P.max(1), bins=20, range=(0, 1), color=c, alpha=0.45, histtype="stepfilled", label=lab)
ax.set_xlim(0.2, 1.01); ax.set_ylim(0.2, 1.01); ax.set_ylabel("Observed accuracy")
ax.legend(loc="upper left", fontsize=8)
ax.set_title("Reliability of the predicted stage's probability", fontweight="bold")
axh.set_xlabel("Predicted probability of the chosen stage"); axh.set_ylabel("Patients")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig4_calibration.png"); plt.show()
```

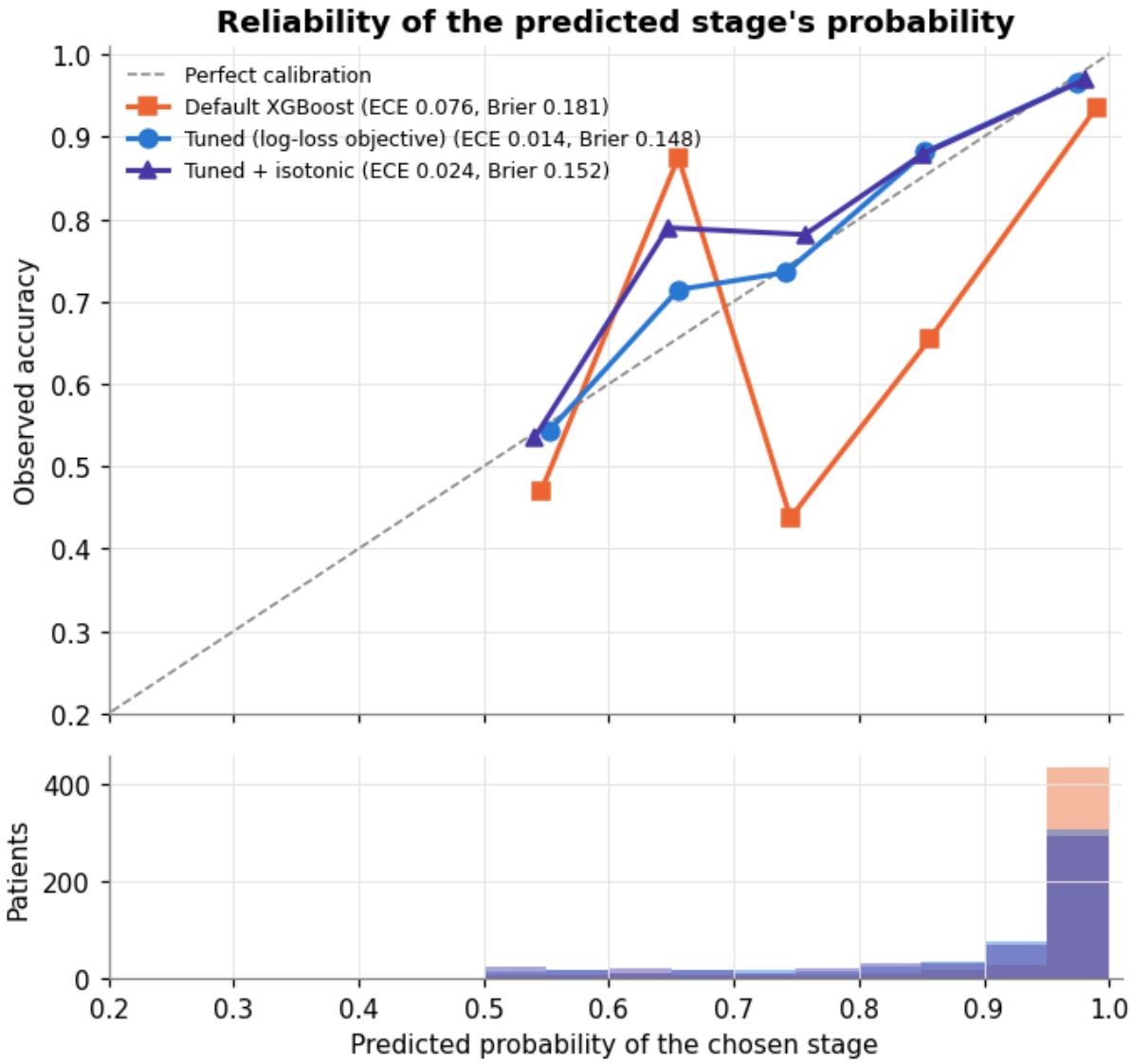
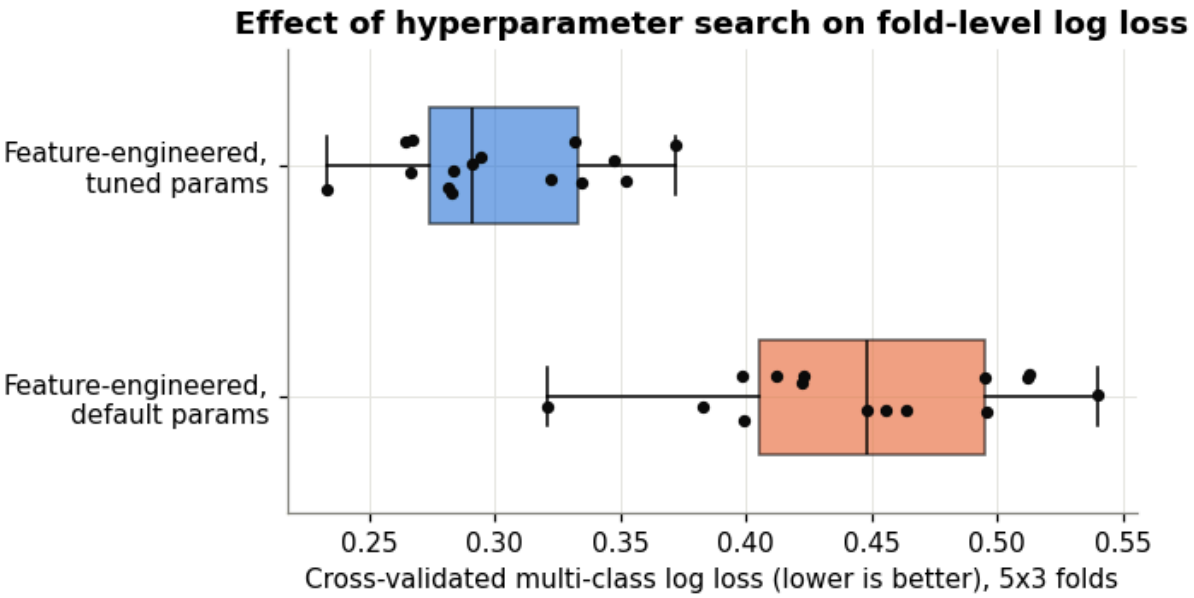


Figure 5: Cross-validation stability before and after hyperparameter search

In [14]:

```
fig, ax = plt.subplots(figsize=(6.4, 3.4))
data = [-cv_before, -cv_after]
bp = ax.boxplot(data, vert=False, widths=0.5, patch_artist=True, medianprops=dict(color="#0b0b0b"))
for patch, c in zip(bp["boxes"], [BEFORE, AFTER]):
    patch.set_facecolor(c); patch.set_alpha(0.6)
for k, d in enumerate(data):
    ax.scatter(d, np.full(d.size, k + 1) + rng.uniform(-0.12, 0.12, d.size), s=14, color="#0b0b0b", zorder=3)
ax.set_yticks([1, 2], ["Feature-engineered,\ndefault params", "Feature-engineered,\ntuned params"])
ax.set_xlabel("Cross-validated multi-class log loss (lower is better), 5x3 folds")
ax.set_title("Effect of hyperparameter search on fold-level log loss", fontweight="bold")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig5_cv_stability.png"); plt.show()
```



7. Exploratory data and trend analysis

Figure 6: 24-hour systolic profiles by stage (mean with 95% CI), computed from the readings

In [15]:

```
prof = abpm.merge(patients[["pid", "target"]], on="pid").groupby(["target", "hour"]).sbp
mean, sem = prof.mean().unstack(0), prof.sem().unstack(0)
fig, ax = plt.subplots(figsize=(9, 4.4))
ax.axvspan(0, 6, color="#8a8984", alpha=0.10, lw=0); ax.axvspan(22.5, 24, color="#8a8984", alpha=0.10, lw=0)
ax.axvspan(6, 10, color="#eda100", alpha=0.15, lw=0)
for i in range(4):
    m, e = mean[i], 1.96 * sem[i]
    ax.fill_between(m.index, m - e, m + e, color=STAGE_COLORS[i], alpha=0.25, lw=0)
    ax.plot(m.index, m, color=STAGE_COLORS[i], lw=2, label=STAGES[i])
    ax.text(23.6, m.iloc[-1], STAGES[i], fontsize=8, va="center", color="#52514e")
ax.text(3, ax.get_ylim()[0] + 1.5, "typical sleep", ha="center", fontsize=8, color="#52514e")
ax.text(8, ax.get_ylim()[0] + 1.5, "morning window (06:00-10:00)", ha="center", fontsize=8, color="#52514e")
ax.set_xlim(0, 25.5); ax.set_xticks(range(0, 25, 2), [f"{h:02d}:00" for h in range(0, 25, 2)], fontsize=8)
ax.set_xlabel("Clock time"); ax.set_ylabel("Systolic BP (mmHg)")
ax.set_title(f"Mean half-hourly systolic BP by stage (n = {len(patients):,} patients)", fontweight="bold")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig6_circadian.png"); plt.show()
```

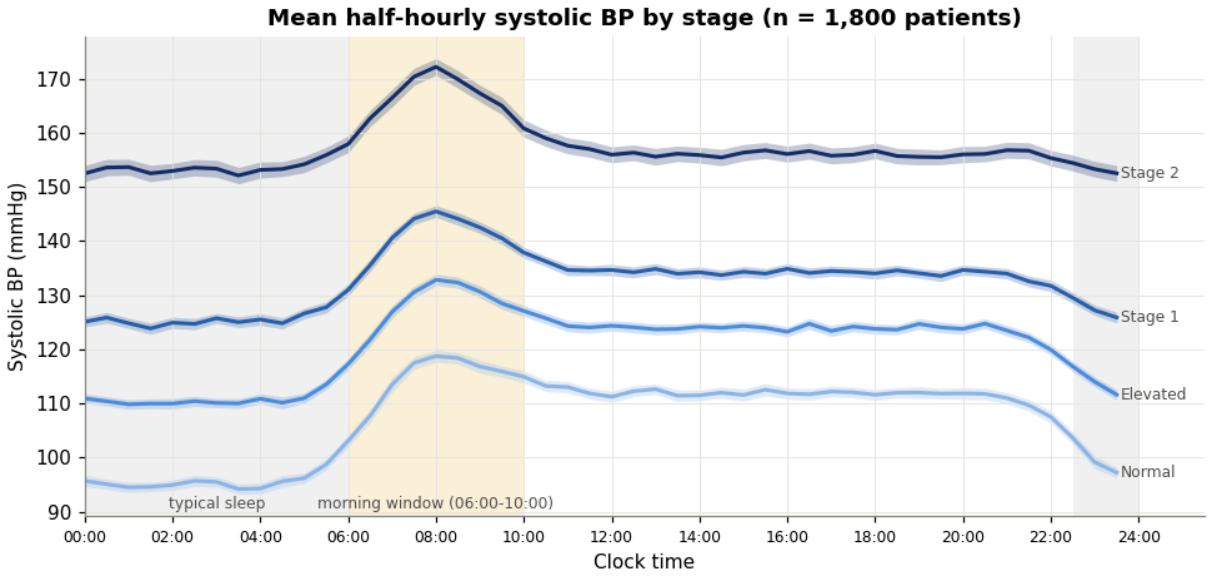
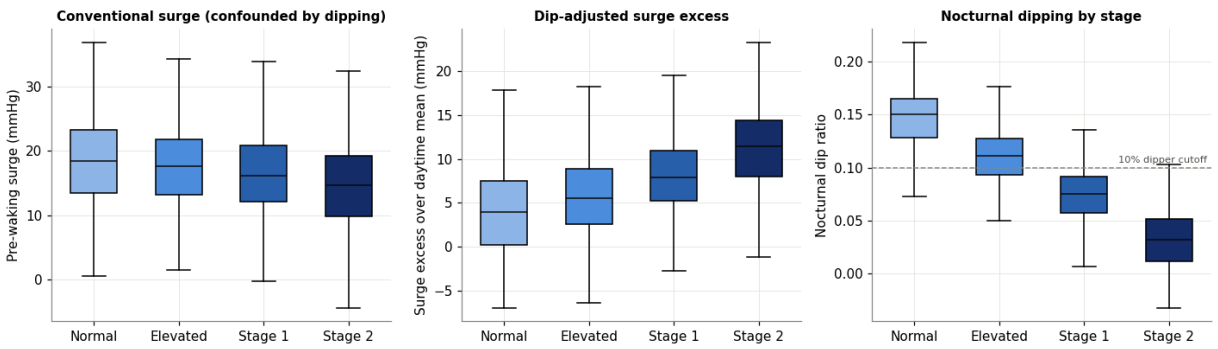


Figure 7: Morning surge and nocturnal dipping by stage

In [16]:

```
fig, (a1, a3, a2) = plt.subplots(1, 3, figsize=(13, 3.8))
for ax, col, lab in [(a1, "surge_prewake", "Pre-waking surge (mmHg)"),
                     (a3, "surge_excess", "Surge excess over daytime mean (mmHg)"),
                     (a2, "dip_ratio", "Nocturnal dip ratio")]:
    d = [df.loc[df.target == i, col].dropna() for i in range(4)]
    bp = ax.boxplot(d, widths=0.55, patch_artist=True, showfliers=False,
                    medianprops=dict(color="#0b0b0b"))
    for patch, c in zip(bp["boxes"], STAGE_COLORS):
        patch.set_facecolor(c)
    ax.set_xticks(range(1, 5), STAGES); ax.set_ylabel(lab)
a2.axhline(0.10, ls="--", color="#8a8984", lw=1); a2.text(4.45, 0.105, "10% dipper cutoff", fontsize=7.5, ha="right",
color="#52514e")
a1.set_title("Conventional surge (confounded by dipping)", fontweight="bold", fontsize=10)
a3.set_title("Dip-adjusted surge excess", fontweight="bold", fontsize=10)
a2.set_title("Nocturnal dipping by stage", fontweight="bold", fontsize=10)
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig7_surge_dip.png"); plt.show()

eda = {}
eda["surge_prewake_median"] = df.groupby("target").surge_prewake.median().round(1).tolist()
eda["surge_trough_median"] = df.groupby("target").surge_sleep_trough.median().round(1).tolist()
eda["surge_excess_median"] = df.groupby("target").surge_excess.median().round(1).tolist()
eda["dip_median"] = df.groupby("target").dip_ratio.median().round(3).tolist()
eda["nondipper_share"] = (df.assign(nd=df.dip_ratio < 0.10).groupby("target").nd.mean()).round(3).tolist()
eda
```

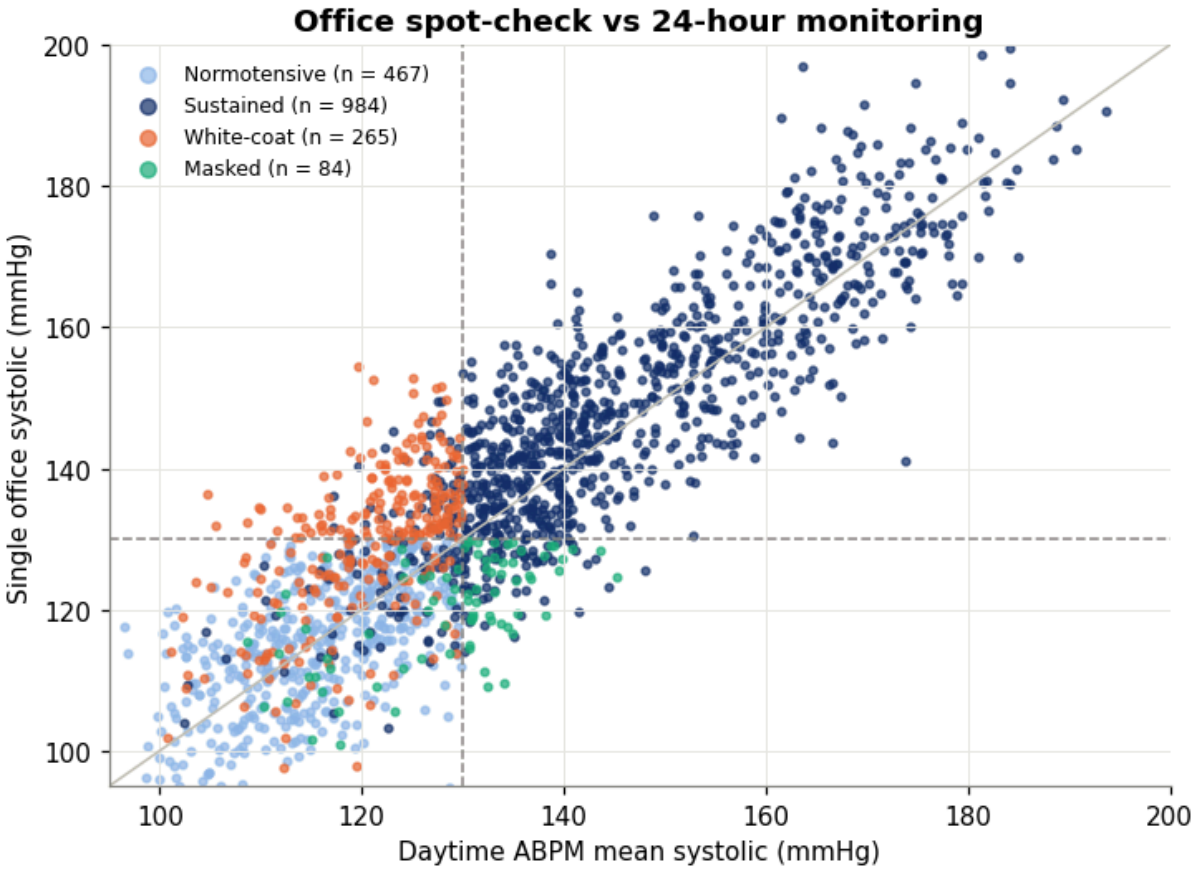


Out[16]:
```
{'surge_prewake_median': [18.5, 17.6, 16.2, 14.6],
 'surge_trough_median': [25.8, 24.0, 22.6, 21.3],
 'surge_excess_median': [4.0, 5.5, 7.9, 11.4],
 'dip_median': [0.15, 0.111, 0.075, 0.032],
 'nondipper_share': [0.051, 0.329, 0.833, 0.993]}
```

Figure 8: Office spot-check versus ABPM: white-coat and masked patterns

Each patient's single office reading is compared with their daytime ABPM mean using the 2017 guideline correspondence (office ≥130/80 ↔ daytime ≥130/80). **White-coat**: office says hypertensive, ABPM does not. **Masked**: ABPM says hypertensive, office does not.

In [17]:
```
office_htn = (df.office_sbp >= 130) | (df.office_dbp >= 80)
abpm_htn = (df.sbp_day >= 130) | (df.dbp_day >= 80)
df["phenotype"] = np.select([office_htn & ~abpm_htn, ~office_htn & abpm_htn, office_htn & abpm_htn],
                            ["White-coat", "Masked", "Sustained"], "Normotensive")
eda["phenotype_counts"] = df.phenotype.value_counts().to_dict()
eda["office_rule_misclassified"] = int((pred_office != y_te).sum())
eda["office_vs_true_htn_disagree"] = float(((df.office_sbp >= 130) | (df.office_dbp >= 80)).ne(df.target >= 2).mean())
fig, ax = plt.subplots(figsize=(6.8, 5))
ph_col = {"Normotensive": "#8fb8ea", "Sustained": "#14306b", "White-coat": "#eb6834", "Masked": "#1baf7a"}
for ph, c in ph_col.items():
    m = df.phenotype == ph
    ax.scatter(df.sbp_day[m], df.office_sbp[m], s=9, color=c, alpha=0.7,
               label=f"{ph} (n = {m.sum()})")
ax.axvline(130, ls="--", color="#8a8984", lw=1); ax.axhline(130, ls="--", color="#8a8984", lw=1)
lim = [95, 200]; ax.plot(lim, lim, color="#c3c2b7", lw=1)
ax.set_xlim(lim); ax.set_ylim(lim)
ax.set_xlabel("Daytime ABPM mean systolic (mmHg)"); ax.set_ylabel("Single office systolic (mmHg)")
ax.legend(fontsize=8, loc="upper left", markerscale=2)
ax.set_title("Office spot-check vs 24-hour monitoring", fontweight="bold")
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig8_office_vs_abpm.png"); plt.show()
eda["phenotype_counts"]
```

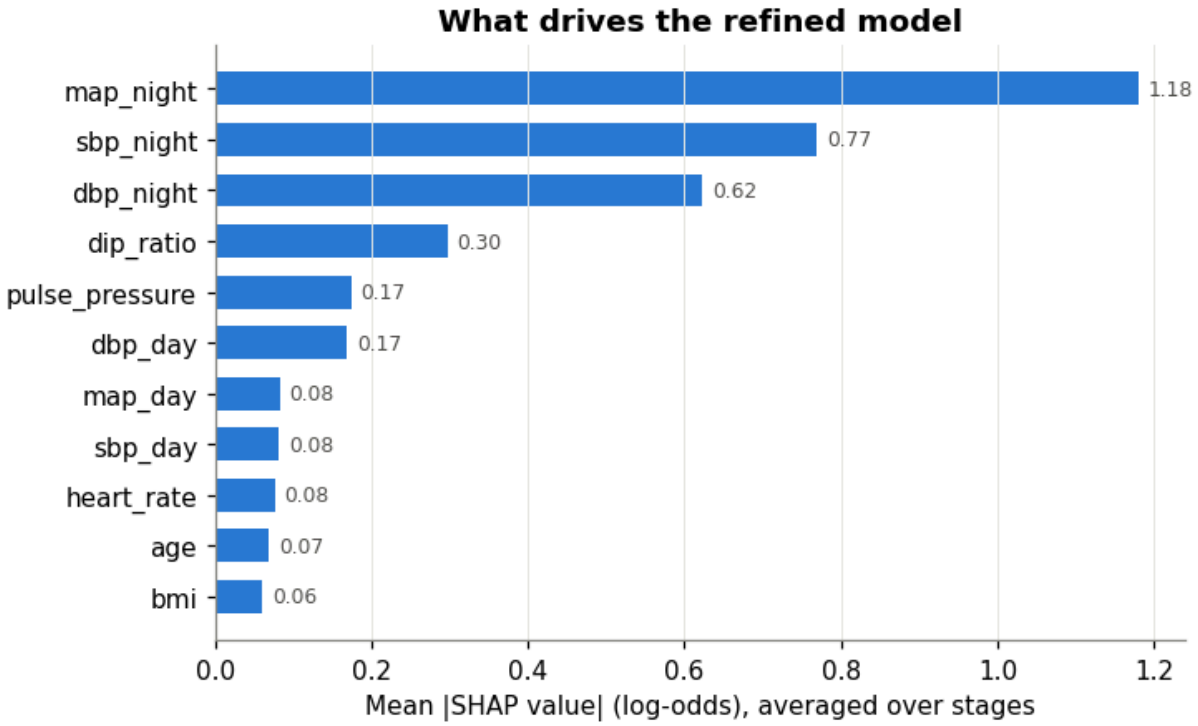


Out[17]: {'Sustained': 984, 'Normotensive': 467, 'White-coat': 265, 'Masked': 84}

Figure 9: Global feature attribution (mean |SHAP|, tuned model, holdout)

In [18]:

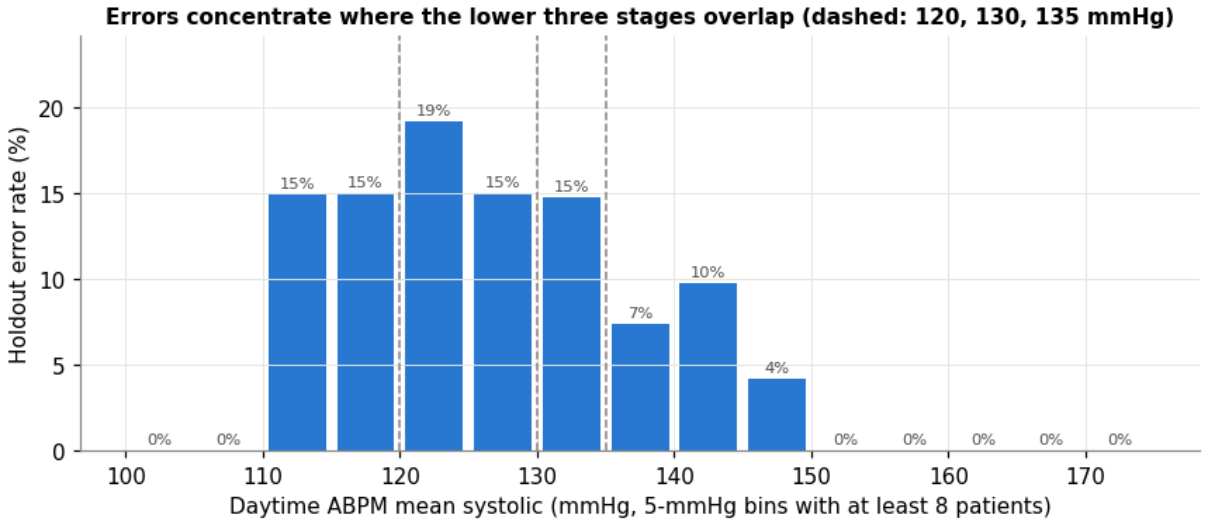
```
sv = explainer.shap_values(te[ENG])
sv = np.stack(sv, -1) if isinstance(sv, list) else sv  # (n, features, classes)
imp = pd.Series(np.abs(sv).mean(axis=(0, 2)), index=ENG).sort_values()
fig, ax = plt.subplots(figsize=(6.8, 4.2))
ax.barh(imp.index, imp.values, color="#2a78d6", height=0.65)
for yv, v in enumerate(imp.values):
    ax.text(v + imp.max() * 0.01, yv, f"{v:.2f}", va="center", fontsize=8, color="#52514e")
ax.set_xlabel("Mean |SHAP value| (log-odds), averaged over stages")
ax.set_title("What drives the refined model", fontweight="bold"); ax.grid(axis="y", visible=False)
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig9_shap.png"); plt.show()
eda["shap_top"] = imp.sort_values(ascending=False).round(3).head(6).to_dict()
eda["shap_top"]
```



Out[18]:
```
{'map_night': 1.180999994277954,
 'sbp_night': 0.7699999809265137,
 'dbp_night': 0.6230000257492065,
 'dip_ratio': 0.296999990940094,
 'pulse_pressure': 0.17299999296665192,
 'dbp_day': 0.16899999976158142}
```

Figure 10: Where the refined model errs (tuned model) (error rate by daytime systolic level)

In [19]:
```
err = pd.DataFrame(dict(sbp=te.sbp_day.values, wrong=(P_tuned.argmax(1) != y_te)))
err["bin"] = pd.cut(err.sbp, np.arange(95, 191, 5))
rate = err.groupby("bin").wrong.agg(["mean", "size"])
rate = rate[rate["size"] >= 8]
mids = [b.mid for b in rate.index]
fig, ax = plt.subplots(figsize=(8, 3.6))
ax.bar(mids, rate["mean"] * 100, width=4.2, color="#2a78d6")
for m_, v in zip(mids, rate["mean"].values):
    ax.text(m_, v * 100 + 0.4, f"{v:.0%}", ha="center", fontsize=7.5, color="#52514e")
for x_ in [120, 130, 135]:
    ax.axvline(x_, ls="--", color="#8a8984", lw=1)
ax.set_xlabel("Daytime ABPM mean systolic (mmHg, 5-mmHg bins with at least 8 patients)")
ax.set_ylabel("Holdout error rate (%)")
ax.set_ylim(0, rate["mean"].max() * 100 + 5)
ax.set_title("Errors concentrate where the lower three stages overlap (dashed: 120, 130, 135 mmHg)", fontweight="bold",
fontsize=10)
plt.tight_layout(); plt.savefig(f"{OUT}/topic6_fig10_errors.png"); plt.show()
near = (np.abs(te.sbp_day.values - 130) <= 5) | (np.abs(te.sbp_day.values - 120) <= 5)
eda["err_near_boundary"] = float(err.wrong[near].mean()); eda["err_far"] = float(err.wrong[~near].mean())
eda["share_errors_near_boundary"] = float(err.wrong[near].sum() / max(err.wrong.sum(), 1))
eda["err_near_boundary"], eda["err_far"], eda["share_errors_near_boundary"]
```



Out[19]: (0.1615720524017467, 0.05144694533762058, 0.6981132075471698)

Follow-up experiment: does a morning-surge feature help?

The exploratory analysis suggests adding the surge features. They are tested here with the *same* tuned hyperparameters, so any change is attributable to the features.

In [20]:

```
xgb_surge = clone(xgb_tuned).fit(tr[SURGE], y_tr)
P_surge = xgb_surge.predict_proba(te[SURGE])
res["xgb_surge"] = evaluate("+ Morning-surge features", P_surge.argmax(1), P_surge)
cvs = cross_val_score(clone(xgb_tuned), tr[SURGE], y_tr, cv=rcv, scoring="neg_log_loss", n_jobs=1)
eda["surge_experiment"] = dict(holdout={k: round(res["xgb_surge"][k], 4) for k in
                                        ["accuracy", "macro_f1", "crisis_recall", "brier", "ece"]},
                               cv_logloss=[float(-cvs.mean()), float(cvs.std())])
eda["surge_experiment"]
```

Out[20]:

```
{'holdout': {'accuracy': 0.9056,
  'macro_f1': 0.9059,
  'crisis_recall': 0.9704,
  'brier': 0.1478,
  'ece': 0.02},
 'cv_logloss': [0.29885154476631776, 0.0379505856729736]}
```

Subgroup check (sex and age band)

In [21]:

```
sub = te.assign(correct=(P_tuned.argmax(1) == y_te),
                age_band=pd.cut(te.age, [0, 45, 65, 120], labels=["<45", "45-64", "65+"]))
ci95 = lambda p, n: 1.96 * np.sqrt(p * (1 - p) / n)
subgroups = {f"sex={k}": [float(v.mean()), int(v.size), float(ci95(v.mean(), v.size))] for k, v in sub.groupby("sex").
correct}
subgroups.update({f"age={k}": [float(v.mean()), int(v.size), float(ci95(v.mean(), v.size))] for k, v in sub.groupby("a
ge_band").correct})
eda["subgroups"] = subgroups
subgroups
```

Out[21]:
```
{'sex=F': [0.9068100358422939, 279, 0.034111176693743445],
 'sex=M': [0.896551724137931, 261, 0.03694750538749816],
 'age=<45': [0.8602150537634409, 93, 0.0704771475160658],
 'age=45-64': [0.8978328173374613, 323, 0.033029975895090664],
 'age=65+': [0.9435483870967742, 124, 0.040622392165634874]}
```

8. Save metrics for the report

In [22]:
```
metrics.update(results=res, eda=eda, n_train=int(len(tr)), n_test=int(len(te)),
               test_class_counts=np.bincount(y_te).tolist())
with open(f"{OUT}/topic6_metrics.json", "w") as f:
    json.dump(metrics, f, indent=2, default=lambda o: o.tolist() if hasattr(o, "tolist") else str(o))
summary.astype(float).round(3)
```

Out[22]:

name	accuracy	macro_f1	crisis_recall	macro_roc_auc	brier	ece	latency_ms
Office AHA rule (single reading)	0.543	0.521	0.956	NaN	NaN	NaN	0.224
ABPM guideline rule (daytime mean)	0.615	0.588	0.993	NaN	NaN	NaN	0.224
Logistic regression	0.883	0.884	0.948	0.980	0.182	0.056	2.345
Default XGBoost	0.889	0.889	0.970	0.980	0.181	0.076	2.904
+ Feature engineering	0.889	0.889	0.970	0.980	0.176	0.065	3.854
+ Hyperparameter search	0.902	0.902	0.970	0.984	0.148	0.014	3.547
+ Isotonic calibration	0.900	0.900	0.970	0.982	0.152	0.024	22.673

References

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Kario, K., Pickering, T. G., Umeda, Y., Hoshide, S., Hoshide, Y., Morinari, M., Murata, M., Kuroda, T., Schwartz, J. E., & Shimada, K. (2003). Morning surge in blood pressure as a predictor of silent and clinical cerebrovascular disease in elderly hypertensives. *Circulation, 107*(10), 1401–1406. https://doi.org/10.1161/01.CIR.0000056521.67546.AA

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence, 2*(1), 56–67. https://doi.org/10.1038/s42256-019-0138-9

Whelton, P. K., Carey, R. M., Aronow, W. S., et al. (2018). 2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults. *Journal of the American College of Cardiology, 71*(19), e127–e248. https://doi.org/10.1016/j.jacc.2017.11.006